TECHNICAL STANDARD OPERATING PROCEDURE

Connecting pgAdmin 4 (Windows) to PostgreSQL 18 on an Ubuntu 24.04 VM

Configuring and validating remote TCP/IP connectivity between a Windows pgAdmin 4 client and a PostgreSQL 18 server hosted in a VMware Workstation virtual machine

Document Type	Technical SOP
Technology Stack	PostgreSQL 18, Ubuntu 24.04, VMware Workstation, pgAdmin 4
Author	Praveen Madupu
Date	Monday — 10th August 2026
Status	Final — Validated
Classification	Internal / Lab Environment

Table of Contents

1. Objective 3
2. Architecture 3
3. Prerequisites 4
4. Verify the PostgreSQL Cluster 4
5. Verify the PostgreSQL Service 5
6. Verify PostgreSQL IP Configuration 5
7. Restart PostgreSQL 5
8. Verify the PostgreSQL Port 6
9. Configure pg_hba.conf 6
10. Allow Remote LAN Connections 7
11. Reload PostgreSQL 7
12. Validate pg_hba.conf 7
13. Configure the Ubuntu Firewall 7
14. Allow PostgreSQL From the Windows Client 8
15. Test Network Connectivity 8
16. Configure PostgreSQL Password Authentication 9
17. pgAdmin 4 Configuration 9
18. Authentication Error Encountered 10
19. Final Working Configuration 11
20. Troubleshooting Decision Tree 11
21. Common Errors and Their Meaning 12
22. Useful Validation Commands 12
23. Security Best Practices 13
24. Official PostgreSQL References 14
25. Final SOP Summary 14

1. Objective

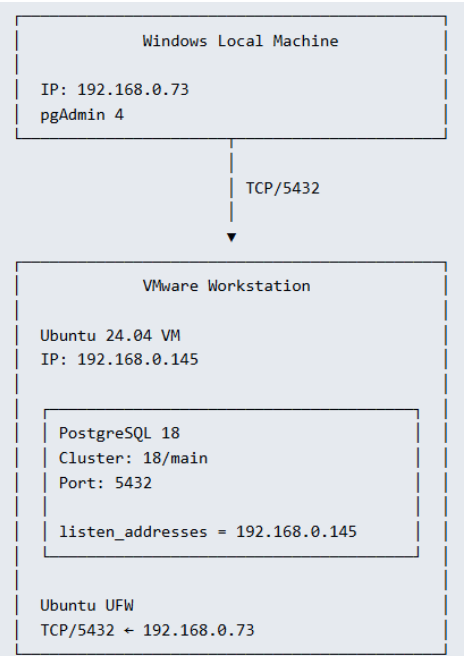
This SOP explains how to configure a PostgreSQL 18 server running on an Ubuntu 24.04 VMware virtual machine so that pgAdmin 4, running on a Windows workstation, can connect to it remotely over TCP/IP.

1.1 Environment Summary

Component	Details
Client OS	Windows
Client Tool	pgAdmin 4
Windows IP	192.168.0.73
Virtualization	VMware Workstation
Database Server OS	Ubuntu 24.04
PostgreSQL Version	PostgreSQL 18
PostgreSQL Cluster	18/main
PostgreSQL Server IP	192.168.0.145
PostgreSQL Port	5432
Database User	postgres
Authentication Method	scram-sha-256

2. Architecture

The client (Windows / pgAdmin 4) reaches the PostgreSQL server over TCP port 5432, through VMware's virtual network and the Ubuntu host's firewall (UFW):



3. Prerequisites

Before starting, verify the following:

- Ubuntu 24.04 VM is running.
- PostgreSQL 18 is installed.
- PostgreSQL cluster 18/main exists.
- VM has IP 192.168.0.145.
- Windows machine can reach the VM.
- pgAdmin 4 is installed on Windows.
- PostgreSQL postgres user password is known.
- VMware networking permits communication between Windows and the VM.

4. Verify the PostgreSQL Cluster

On Ubuntu, run:

```
pg_lsclusters
```

Expected output:

```
Ver Cluster Port Status Owner    Data directory
18  main    5432 online postgres /var/lib/postgresql/18/main
```

This confirms the PostgreSQL 18 cluster is:

```
Version : 18
Cluster : main
Port    : 5432
Status  : online
```

5. Verify the PostgreSQL Service

Run:

```
systemctl status postgresql
```

The Ubuntu PostgreSQL packaging uses a general postgresql.service, plus cluster-specific services such as postgresql@18-main.service. Verify the actual cluster:

```
systemctl status postgresql@18-main
```

Expected:

```
Active: active (running)
```

6. Verify PostgreSQL IP Configuration

Initially, the server was configured with listen_addresses = localhost. This allows local TCP/IP connections but does not allow PostgreSQL to listen on the VM's network interface.

Check the current value:

```
sudo -u postgres psql -c "SHOW listen_addresses;"
```

Initial state

```
localhost
```

Required configuration

Edit the configuration file:

```
sudo nano /etc/postgresql/18/main/postgresql.conf
```

Set:

```
listen_addresses = '192.168.0.145'
```

> **Why this matters**
> PostgreSQL must listen on the VM's network interface so remote clients such as pgAdmin can establish a TCP connection. The listen_addresses parameter controls the TCP/IP addresses on which the server listens.

7. Restart PostgreSQL

Because listen_addresses is a server-start parameter, PostgreSQL must be restarted (a reload is not sufficient):

```
sudo systemctl restart postgresql@18-main
```

Verify:

```
sudo -u postgres psql -c "SHOW listen_addresses;"
```

Expected:

```
 listen_addresses
------------------
 192.168.0.145
```

8. Verify the PostgreSQL Port

```
sudo -u postgres psql -c "SHOW port;"
```

Expected:

```
5432
```

Also verify the operating-system listener:

```
sudo ss -lntp | grep 5432
```

Expected result indicates PostgreSQL is listening on:

```
192.168.0.145:5432
```

9. Configure pg_hba.conf

PostgreSQL uses pg_hba.conf to control client authentication. The configuration file is located at:

```
/etc/postgresql/18/main/pg_hba.conf
```

View it:

```
sudo cat /etc/postgresql/18/main/pg_hba.conf
```

Existing local rules

```
local   all             postgres                                peer
local   all             all                                     peer

host    all             all             127.0.0.1/32            scram-sha-256
host    all             all             ::1/128                 scram-sha-256
```

These rules primarily cover local connections.

10. Allow Remote LAN Connections

For the lab environment, the following rule was added:

```
host    all    all    192.168.0.0/24    scram-sha-256
```

This allows clients from the 192.168.0.0/24 subnet to authenticate using SCRAM.

Security improvement

Because the actual Windows client IP is 192.168.0.73, a more restrictive production-style rule could be used instead:

```
host    all    all    192.168.0.73/32    scram-sha-256
```

/32 means one specific IPv4 address. For this lab, the existing /24 rule worked correctly.

11. Reload PostgreSQL

After changing pg_hba.conf, reload PostgreSQL:

```
sudo systemctl reload postgresql@18-main
```

A complete restart is not required for a pg_hba.conf change alone.

12. Validate pg_hba.conf

PostgreSQL provides the pg_hba_file_rules view. Run:

```
sudo -u postgres psql -c "SELECT line_number, type, database, user_name, address,
  auth_method, error FROM pg_hba_file_rules;"
```

Expected relevant entry:

```
line_number | type | database | user_name | address     | auth_method  | error
------------+------+----------+-----------+-------------+--------------+------
135         | host | {all}    | {all}     | 192.168.0.0 | scram-sha-256|
```

The important point is that error = NULL, confirming PostgreSQL successfully parsed the rule.

13. Configure the Ubuntu Firewall

The initial UFW configuration was:

```
Status: active
Default: deny (incoming)
```

Existing rules included:

```
27017    ALLOW IN    192.168.0.73
22       ALLOW IN    192.168.0.73
```

Root cause identified here

There was initially no rule for PostgreSQL port 5432. Even though PostgreSQL was listening correctly, Ubuntu's firewall blocked the connection.

14. Allow PostgreSQL From the Windows Client

Because only the Windows machine needs access, the recommended firewall rule is:

```
sudo ufw allow from 192.168.0.73 to any port 5432 proto tcp
```

Verify:

```
sudo ufw status numbered
```

Expected:

```
22/tcp     ALLOW IN    192.168.0.73
27017      ALLOW IN    192.168.0.73
5432/tcp   ALLOW IN    192.168.0.73
```

Security principle

Avoid unnecessarily opening 5432 from anywhere. Do not use:

```
sudo ufw allow 5432
```

when only one client requires access. Restricting the source IP reduces the attack surface.

15. Test Network Connectivity

From Windows PowerShell:

```
Test-NetConnection 192.168.0.145 -Port 5432
```

Initial result

```
PingSucceeded    : True
TcpTestSucceeded : False
```

This was an important troubleshooting result. It proved that Windows → VM connectivity was working, but Windows → VM TCP/5432 was blocked.

Root cause

Ubuntu UFW was blocking port 5432. After adding the firewall rule, the test returned:

```
ComputerName     : 192.168.0.145
RemoteAddress    : 192.168.0.145
RemotePort       : 5432
SourceAddress    : 192.168.0.73
TcpTestSucceeded : True
```

This confirmed that the network path was fully operational.

16. Configure PostgreSQL Password Authentication

The pg_hba.conf rule uses scram-sha-256, so the PostgreSQL user needs a valid password.

To reset the postgres user's password:

```
sudo -u postgres psql
```

Then, inside the psql prompt:

```
\password postgres
```

Enter the new password twice, then exit:

```
\q
```

Verify the role:

```
sudo -u postgres psql -c "SELECT rolname, rolcanlogin, rolsuper FROM pg_roles WHERE
rolname = 'postgres';"
```

Expected:

```
rolname  | rolcanlogin | rolsuper
---------+-------------+---------
postgres | t           | t
```

17. pgAdmin 4 Configuration

On the Windows machine, navigate to:

```
pgAdmin 4 → Servers → Register → Server
```

General tab

```
Name: PostgreSQL 18 - Ubuntu VM
```

Connection tab

Parameter	Value
Host name/address	192.168.0.145
Port	5432
Maintenance database	postgres
Username	postgres
Password	PostgreSQL postgres password

Click Save to register the connection.

18. Authentication Error Encountered

During testing, pgAdmin initially returned:

```
FATAL: password authentication failed for user "postgres"
```

At this stage, the diagnostic status was:

Check	Status
Network connectivity	✅ OK
Port 5432	✅ OK
PostgreSQL listener	✅ OK
pg_hba.conf	✅ OK
UFW	✅ OK
Authentication	❌ Failed

Resolution

The postgres password was reset using:

```
sudo -u postgres psql
```

followed by:

```
\password postgres
```

The new password was then entered into pgAdmin. The connection succeeded.

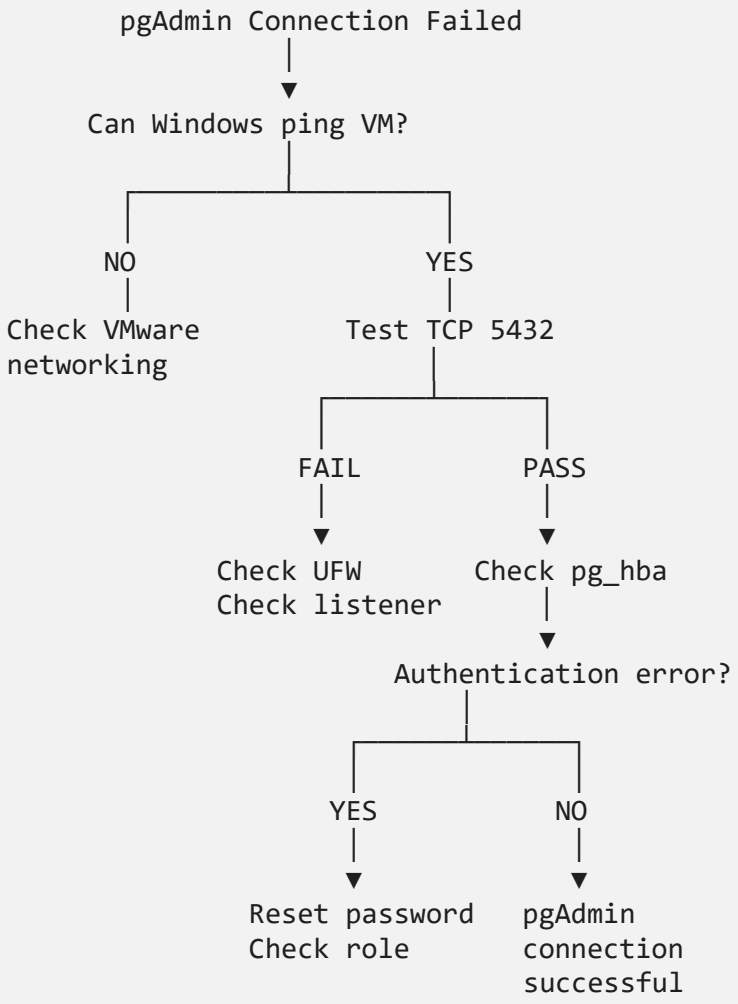
19. Final Working Configuration

Item	Final Value
PostgreSQL version	18
Cluster	18/main
Server IP	192.168.0.145
Port	5432
postgresql.conf — listen_addresses	'192.168.0.145'
pg_hba.conf rule	host all all 192.168.0.0/24 scram-sha-256
UFW rule	5432/tcp ALLOW IN 192.168.0.73
Windows client IP	192.168.0.73

Item	Final Value
pgAdmin host	192.168.0.145
pgAdmin port	5432
pgAdmin database	postgres
pgAdmin user	postgres

20. Troubleshooting Decision Tree

Use this sequence whenever pgAdmin cannot connect:



21. Common Errors and Their Meaning

Error	Likely Cause	Action
Connection timed out	Network/firewall	Check VMware/UFW
Connection refused	PostgreSQL not listening	Check ss -lntp
no pg_hba.conf entry	Client not authorized	Check pg_hba.conf
password authentication failed	Incorrect password	Reset PostgreSQL password
database does not exist	Incorrect database name	Check database
role does not exist	Incorrect username	Check pg_roles
TcpTestSucceeded: False	TCP port inaccessible	Check listener/UFW
TcpTestSucceeded: True	Network path works	Move to PostgreSQL authentication

22. Useful Validation Commands

Check clusters

```
pg_lsclusters
```

Check PostgreSQL service

```
systemctl status postgresql@18-main
```

Check listener

```
sudo ss -lntp | grep 5432
```

Check listening addresses

```
sudo -u postgres psql -c "SHOW listen_addresses;"
```

Check port

```
sudo -u postgres psql -c "SHOW port;"
```

Check HBA configuration

```
sudo -u postgres psql -c "SELECT line_number, type, database, user_name, address,
auth_method, error FROM pg_hba_file_rules;"
```

Check firewall

```
sudo ufw status verbose
```

Check PostgreSQL roles

```
sudo -u postgres psql -c "SELECT rolname, rolcanlogin, rolsuper FROM pg_roles;"
```

Test from Windows

```
Test-NetConnection 192.168.0.145 -Port 5432
```

23. Security Best Practices

For a learning environment, the current configuration is appropriate. For production, harden it further.

1. Restrict pg_hba.conf

Instead of 192.168.0.0/24, prefer a specific trusted client or application subnet where practical. For the current Windows client:

```
host    all    all    192.168.0.73/32    scram-sha-256
```

2. Restrict UFW

Use a source-restricted rule:

```
sudo ufw allow from 192.168.0.73 to any port 5432 proto tcp
```

rather than allowing the entire internet.

3. Don't use trust authentication

Avoid configurations such as:

```
host all all 0.0.0.0/0 trust
```

4. Don't expose PostgreSQL directly to the internet

Do not open port 5432 globally unless there is a specific, secured architecture requiring it.

5. Avoid using postgres for applications

postgres is a superuser role. Applications should use dedicated roles with the minimum required privileges.

6. Consider TLS

For production or untrusted networks, configure PostgreSQL SSL/TLS with appropriate certificate validation.

24. Official PostgreSQL References

The key PostgreSQL concepts used in this configuration are documented in the official PostgreSQL documentation:

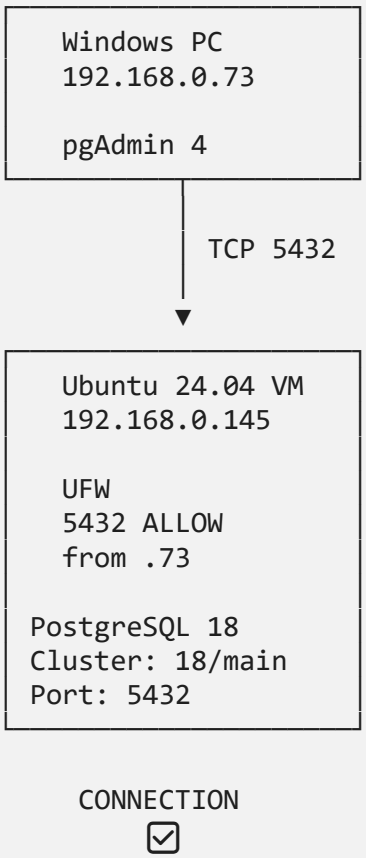
- PostgreSQL 18 — Connections and Authentication — https://www.postgresql.org/docs/18/client-authentication.html
- PostgreSQL 18 — pg_hba.conf — https://www.postgresql.org/docs/18/auth-pg-hba-conf.html
- PostgreSQL 18 — Connection Settings / listen_addresses — https://www.postgresql.org/docs/18/runtime-config-connection.html
- PostgreSQL 18 — Database Roles — https://www.postgresql.org/docs/18/user-manag.html
- PostgreSQL 18 — SSL/TLS Encryption — https://www.postgresql.org/docs/18/ssl-tcp.html

25. Final SOP Summary

The complete troubleshooting and configuration sequence was:

```
1. Verify PostgreSQL 18 cluster
        ↓
2. Identify VM IP = 192.168.0.145
        ↓
3. Change listen_addresses
        ↓
4. Restart PostgreSQL
        ↓
5. Configure pg_hba.conf
        ↓
6. Reload PostgreSQL
        ↓
7. Validate pg_hba_file_rules
        ↓
8. Check Ubuntu UFW
        ↓
9. Allow TCP/5432 from 192.168.0.73
        ↓
10. Test Windows TCP connectivity
        ↓
11. Reset postgres password
        ↓
12. Configure pgAdmin 4
        ↓
13. Successfully connect
```

Final result



Root cause encountered

Ubuntu UFW had a default deny (incoming) policy and did not initially allow TCP/5432.

Secondary authentication issue

The postgres password used by pgAdmin was incorrect; resetting the PostgreSQL role password resolved the authentication failure.

Final outcome

Windows pgAdmin 4 successfully connects remotely to PostgreSQL 18 running on the Ubuntu 24.04 VMware VM.